CHAPTER 27

Request Parameters and Sampling Controls

In this chapter

Introduction: The Parameter That Type-Checked and Failed . . . 3
27.1 The Body, Field by Field . . . 4
27.2 What the Sampling Parameters Actually Did . . . 5
27.3 Temperature Is a Control on Entropy . . . 6
27.4 Top-k Is a Fixed Cut; Top-p Is Not . . . 7
27.5 The Two Parameters Do Not Commute . . . 8
27.6 The Parameters Are Gone . . . 8
27.7 What You Actually Lost . . . 9
27.8 Why temperature = 0 Was Never a Guarantee . . . 9
27.9 max_tokens Is a Ceiling on Thinking Plus Text . . . 10
27.10 output_config: Effort, Task Budget, Format . . . 10
27.11 The Remaining Controls . . . 11
27.12 Visual Analogies . . . 11
27.13 Architecture: Where a Parameter Takes Effect . . . 12
27.14 The Determinism Measurement . . . 13
27.15 Examples . . . 14
27.16 Code Walkthrough: Temperature in Four Lines . . . 15
27.17 Mini Project: The Parameter Bench . . . 16
27.18 Panel A — Temperature, Measured . . . 17
27.19 Panel B — Truncation, and Why One of These Ports . . . 17
27.20 Panel C — Zero Flips in Twenty Thousand . . . 18
27.21 Panel D — max_tokens Against Thinking . . . 19
27.22 Panel E — Twenty-Three Rejections, Six Acceptances, Four Unknowns . . . 19
27.23 Evaluation and Deployment . . . 21
27.24 Hands-on Labs . . . 21
27.25 Lab Solutions . . . 22
27.26 Best Practices . . . 22
27.27 Common Mistakes . . . 24
27.28 Performance Considerations . . . 26

27.29 Security Considerations 26
27.30 Real Production Examples 27
27.31 Certification Notes 27
27.32 Twenty Practice Questions 28
27.33 Answers and Explanations 29
27.34 Ten Practical Exercises 30
27.35 Five Challenge Labs 30
27.36 Ten Interview Questions 31
27.37 Summary 31
27.38 Glossary 32
27.39 Further Reading 33

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Name every field in a Messages request body, say which are required, and say which model each optional one is valid on.
2. Explain what `temperature`, `top_p` and `top_k` do to the next-token distribution in terms of the softmax of Chapter 3 and the decode loop of Chapter 4.
3. State precisely why the same `top_p` truncates two distributions to wildly different sizes while the same `top_k` does not, and why that makes one of them portable and the other not.
4. Say which models reject a non-default sampling parameter, why the SDK type-checks anyway, and what the documentation offers as the replacement.
5. Explain why `temperature = 0` never guaranteed reproducibility, and quantify how far from deterministic greedy decoding actually is.
6. Size `max_tokens` as a ceiling on **thinking plus text**, and compute the truncation rate a given cap implies.
7. Configure `output_config` — `effort`, `task_budget` and `format` — against the per-model support matrix rather than by generation.
8. Set the remaining request-level controls — `stop_sequences`, `metadata`, `service_tier`, `speed`, `stream` and `fallbacks` — knowing what each costs and which are model-specific.

NOTE

Prerequisites

Chapter 3 for the softmax and for temperature as a shape control on a distribution — this chapter is that section made quantitative. **Chapter 4** for the decode loop that consumes the distribution one token at a time. **Chapter 13** for effort and the thinking knobs. **Chapter 14** for the per-model matrix and for the rule that an unverified fact is UNKNOWN. **Chapter 16** for the request's required fields and `stop_reason`. **Chapter 18** for `max_tokens` against thinking, and for the configuration being part of the cache prefix. **Chapter 19** for service tiers, fast mode and `inference_geo`. **Chapter 23** for `output_config.format`. **Chapter 26** for headers, betas and the preflight this chapter's checks extend.

Deferred by name: `tools` **and** `tool_choice` get **Chapters 31–35**. `stream` was **Chapter 17** and is only summarised here. `cache_control`, `cache_hint` **and the Batch parameters** are **Chapter 30**. `context_management` in full is **Chapter 54**. `fallbacks` was **Chapter 25**.

Introduction: The Parameter That Type-Checked and Failed

A research team ported a summarisation service from Claude Sonnet 4.5 to Claude Opus 5. The change was one string. Their config had carried `temperature=0` since the service was written, for the reason everybody carries it: they wanted reproducible output, and reproducible output is what temperature zero is for.

The Python type checker was happy. `temperature` is a documented field on the SDK's request type, it takes a float, and zero is a float. Continuous integration was green. The first request in production returned a 400.

The documentation is explicit, and unusually so about the mechanism:

> *"Setting* `temperature`*,* `top_p`*, or* `top_k` *to a non-default value returns a 400 error on Claude Opus 5, the same as on Claude Opus 4.7.* ***The SDK request types still define these fields for compatibility with earlier models, so code that sets them type-checks, but the API rejects the request server-side.****"*

This is Chapter 23's discovery again, from the other direction. There, an SDK silently *rewrote* a schema so the bytes on the wire were not the bytes you sent. Here an SDK faithfully sends a field the server will not accept, and the type system — the thing you bought the SDK for — cannot tell you. Chapter 26 called this class of surprise *a convenience that decided something on your behalf*. Add a sixth member to that family: **a type that is not a contract.**

The team's second discovery was worse than the first, and it is the reason this chapter spends a section on floating-point arithmetic. When they removed `temperature=0` and re-ran their regression suite, the outputs were not reproducible before either. They had believed for three years in a guarantee that had never existed.

Section 27.14 measures how far from deterministic greedy decoding actually is, and the result is in two halves, both worth having. The **negative** half first, in this book's tradition: across **20,000 trials**, computing the same logits under two different summation orders produced **zero** argmax flips. The naive worry — that floating-point noise routinely scrambles greedy decoding — is simply not supported. The discrepancy between the two orders averages **7.11 × 10** logit units and never exceeded **5.01 × 10** , while the median gap between the best and second-best token is **0.1976**, five orders of magnitude larger.

The **positive** half is what the same numbers say about a long generation. Near zero the top-two gap has a measured density of **3.345 per unit of logit**, so the probability that any given token is close enough to flip is **2.377 × 10** — about one in 420,000. Per token that is nothing. Over a 128,000-token output it is **26.2 per cent**. A single long generation has better than a one-in-four chance of containing at least one token decided by the order in which a sum was accumulated on a machine you do not control.

So: greedy decoding is reproducible to about six decimal places per token and not reproducible at all across a document. Temperature zero would not have fixed that, it does not exist any more, and the honest replacement is the one Chapters 24 and 25 already built — a regression suite that measures a distribution of outcomes rather than an assertion that two strings are equal.

The rest of the chapter is the body of the request, field by field. Two more measurements anchor it. **The same** `top_p` **is not the same setting**: at `top_p = 0.9`, a confident next-token distribution keeps **2 tokens** and an open-ended one keeps **42,523** — a factor of **21,261** from one number, while `top_k = 40` keeps exactly 40 in both cases. And `max_tokens` **is a ceiling on thinking plus text**, so a cap of 4,000 sized for a 1,200-token answer truncates **53.89 per cent** of requests once adaptive thinking starts spending it.

27.1 The Body, Field by Field

Chapter 16 established the three required fields. Here is the whole surface, with the chapter that owns each.

Field	Required	Owner
`model`	yes	Chapter 14

Field	Required	Owner
`max_tokens`	yes	§27.15
`messages`	yes	Chapter 16
`system`	no	Chapter 21
`temperature`, `top_p`, `top_k`	no	§27.2–§27.13 — **rejected on current models**
`stop_sequences`	no	§27.16
`stream`	no	Chapter 17
`thinking`	no	Chapter 18
`output_config.effort`	no	§27.17
`output_config.task_budget`	no	§27.17
`output_config.format`	no	Chapter 23
`tools`, `tool_choice`	no	Chapters 31–35
`metadata`	no	§27.18
`service_tier`	no	Chapter 19, §27.19
`speed` (fast mode)	no	Chapter 19, §27.19
`inference_geo`	no	Chapter 11
`fallbacks`	no	Chapter 25
`context_management`	no	Chapters 18 and 54
`cache_control`	no	Chapter 30

Three observations before the detail. First, **the required set is tiny** — three fields — and everything else is a decision you are making whether or not you write it down, exactly as Chapter 26 found for `max_retries`. Second, **almost every optional field is model-specific**, which is why this chapter's preflight is a matrix and not a list. Third, **the sampling parameters are the only fields that were removed rather than added**, and understanding what they did is the only way to understand what you now do instead.

27.2 What the Sampling Parameters Actually Did

Chapter 4 ended each decode step with a vector of logits, one per token in the vocabulary, and Chapter 3 turned that vector into a probability distribution with the softmax. Everything in this section happens between those two steps.

DEFINITION

Definition — logits, softmax, temperature

A **logit** is the unnormalised score the model assigns a token. The **softmax** turns a vector of logits z into probabilities, $p = \exp(z) / \exp(z)$. **Temperature** T divides the logits before the softmax: $p \propto \exp(z / T)$. **Top-k** keeps the k highest-probability tokens and renormalises. **Top-p**, or nucleus sampling, keeps the smallest set of tokens whose probabilities sum to at least p, and renormalises.

To make this checkable rather than merely stated, this chapter uses next-token distributions that are exact **power laws in rank**: logits $z = -\cdot\ln(\text{rank})$. Real next-token distributions are approximately power-law in rank, and this shape has a property that turns the whole section into arithmetic you can do on paper.

Because the softmax divides logits by T, and these logits are $-\cdot\ln(\text{rank})$, sampling at temperature T is **exactly** sampling from a power law with exponent $/ T$.

KEY TAKEAWAYS

Temperature is a divisor on the power-law exponent

Halving the temperature doubles the exponent. That is the whole mechanism, and `test_temperature_is_a_divisor_on_the_power_law_exponent` asserts the two distributions are identical to within 10^{12}.

Two distributions carry the chapter. The **peaked** one, $= 3.0$, is *"the capital of France is ___"*: one answer, one plausible alternative, nothing else. The **flat** one, $= 0.35$, is *"write me a story about ___"*: tens of thousands of reasonable continuations. Both have a vocabulary of 50,000.

27.3 Temperature Is a Control on Entropy

"Temperature makes the model more creative" is a slogan, not a specification. The specification is that temperature changes the **entropy** of the next-token distribution, and the honest way to report entropy is as an **effective vocabulary** — 2 raised to the entropy in bits, which is the number of equally likely tokens the distribution behaves like.

T	top-1 p	entropy	effective vocab	/T
Peaked, = 3.0				
0.1	1.0000	0.000 b	1.0	30.00
0.5	0.9830	0.134 b	1.1	6.00
0.7	0.9389	0.398 b	1.3	4.29
1.0	0.8319	0.979 b	2.0	3.00
1.5	0.6079	2.362 b	5.1	2.00
2.0	0.3841	4.554 b	23.5	1.50
Flat, = 0.35				

T	top-1 p	entropy	effective vocab	/T
0.1	0.8875	0.679 b	1.6	3.50
0.5	0.0121	14.321 b	20,464.6	0.70
0.7	0.0022	15.189 b	37,365.8	0.50
1.0	0.0006	15.457 b	44,975.5	0.35
1.5	0.0002	15.554 b	48,117.4	0.23
2.0	0.0001	15.581 b	49,026.6	0.17

Read the two halves against each other and the problem with temperature as a product control becomes obvious. **The same *T* does completely different things depending on what the model already believes.** At $T = 0.5$ the peaked distribution has an effective vocabulary of **1.1** — it is decided — while the flat one has **20,464.6**. A single global setting is being applied to a quantity whose meaning changes token by token.

That is the deep reason a fixed temperature is a poor instrument for controlling behaviour, and it is worth holding on to when the next sections take the instrument away.

27.4 Top-k Is a Fixed Cut; Top-p Is Not

Now truncation. Both parameters restrict the candidate set before sampling. They differ in whether the restriction adapts to the distribution.

Setting	Value	Peaked survivors	Flat survivors	Ratio
`top_p`	0.5	**1**	**17,228**	17,228×
`top_p`	0.9	**2**	**42,523**	**21,261×**
`top_p`	0.95	3	46,209	15,403×
`top_p`	0.99	6	49,234	8,206×
`top_k`	1	1	1	1×
`top_k`	5	5	5	1×
`top_k`	40	40	40	1×
`top_k`	100	100	100	1×

`top_k = 40` keeps exactly forty tokens, always, by definition. `top_p = 0.9` keeps **two** tokens when the model is confident and **forty-two and a half thousand** when it is not.

Which of these is the better parameter depends entirely on what you wanted. If you wanted "never consider a token the model thinks is unlikely", `top_p` is right and `top_k` is wrong, because on a confident distribution `top_k = 40` admits 38 tokens the model has essentially ruled out. If you wanted "the candidate set is always the same size, so my latency and my behaviour are predictable", `top_k` is right

and `top_p` is wrong.

KEY TAKEAWAYS

A setting that adapts is not a setting that transfers

`top_p = 0.9` is not one configuration. It is a different configuration for every prompt, and the difference between its extremes here is a factor of **21,261**. This is why a `top_p` tuned on one workload routinely behaves unrecognisably on another — and why an evaluation set (Chapter 24) was always the only real evidence.

27.5 The Two Parameters Do Not Commute

A third fact, less famous and more awkward: applying temperature and then top-p is not the same as applying top-p and then temperature. Both are "temperature 0.7, top_p 0.9".

Distribution	TV distance between the two orders	Survivors
peaked	**0.0488**	1 vs 2
flat	0.0238	40,530 vs 42,523

Total variation distance is half the L1 distance between two distributions and lives in [0, 1], so 0.0488 is not enormous — but it is not zero, and it changes the *size of the candidate set*, which is the thing you were configuring. Two libraries applying "the same" two numbers in different orders were never running the same experiment.

This is the sort of detail that makes a parameter hard to specify, hard to port and hard to reason about — and it is part of the case for the direction the platform has taken.

27.6 The Parameters Are Gone

> *"The `temperature`, `top_p`, and `top_k` sampling parameters are not supported on Claude 4.7 and later models and Claude Mythos Preview. Setting them to a non-default value returns a 400 error. Omit them from request payloads and use prompting to guide the model's behavior instead."*

That is a **generational rule**, which is easier to hold than a list: **Claude 4.7 and later**, plus Mythos Preview. Enumerated against the models this book tracks, that is Opus 4.7, Opus 4.8, Opus 5, Sonnet 5, Fable 5, Mythos 5 and Mythos Preview.

On **earlier** models the parameters survive, with a narrower restriction that applies only while thinking is on: `temperature` and `top_k` are **incompatible with thinking**, and `top_p` is **allowed only between 0.95 and 1**. With thinking disabled, an older model accepts all three as it always did. Section 27.20 reproduces both regimes.

Three consequences worth stating plainly.

There is no determinism exemption. `temperature = 0` is a non-default value like any other and returns the same 400. Section 27.14 explains why this costs you less than you think.

The SDK will not catch it. The types remain for compatibility with earlier models. This is a runtime failure in a language where you were promised a compile-time one, and the only defence is a preflight — §27.20's, or Chapter 26's, or a test.

The documented replacement is prompting. Not a different parameter: *"use prompting to guide the model's behavior instead."* Chapter 20's principles are the replacement for temperature, Chapter 23's structured outputs are the replacement for the low-temperature-for-format trick, and `effort` (§27.17) is the replacement for the low-temperature-for-brevity trick.

> WARNING
>
> **What "default" means**
>
> The 400 fires on a **non-default** value. Omitting the field entirely is always correct on every model, and — from Chapter 18 — setting a parameter to its default is equivalent to omitting it, which matters because the configuration is part of the cache prefix. Omit them.

27.7 What You Actually Lost

It is worth being honest about the trade rather than merely cheerful.

You lost a cheap creativity dial. For generative work — brainstorming, variation, ideation — a temperature slider was a genuinely useful, zero-latency control, and prompting for variety is wordier and consumes tokens in a way a divisor did not.

You lost the ability to sample the same prompt n times and get a diverse set. Chapter 13's `accuracy ≈ coverage × selection` depended on it: best-of-n needs *n different samples*. Under adaptive thinking, variation now comes from the reasoning path rather than from a sampling parameter you set, which is a less direct instrument.

You did not lose determinism, because you did not have it (§27.14).

You did not lose format control, because constrained decoding (Chapter 23) is strictly better at it: a grammar is a hard guarantee where a low temperature was a hopeful one.

You did not lose brevity control, because `effort` addresses it directly and, per Chapter 15, *"tuning effort is often a better lever than switching models."*

Net: two real losses, both in the generative-diversity direction, and two apparent losses that were illusions. That is a defensible trade, and it is a much better summary than either "they took my parameters away" or "you never needed them".

27.8 Why `temperature = 0` Was Never a Guarantee

Greedy decoding — always take the argmax — is deterministic **in exact arithmetic**. Argmax is a function; the same input gives the same output. The arithmetic is not exact.

Each logit is a dot product over the model's hidden dimension: thousands of floating-point multiplications summed together. Floating-point addition is **not associative**: (a + b) + c and a + (b + c) can differ in the last bits. So the same dot product, summed in a different order, gives a slightly different

number. And the summation order changes for reasons entirely outside your request — a different batch size on the server, a different kernel, a different accelerator, a different partitioning of the reduction.

When the top two logits are close enough together, that last-bit difference flips the argmax. One different token early in a generation changes everything after it, because Chapter 4's decode loop feeds each token back in.

Section 27.14 measures all of it. The headline is that the effect is far smaller than folklore suggests and far from absent, and both halves matter.

27.9 `max_tokens` Is a Ceiling on Thinking Plus Text

`max_tokens` is required and must be at least 1. It is not a hint. Two properties make it the parameter most often set wrongly.

It is a ceiling on total output. The migration documentation is explicit: *"`max_tokens` remains a hard limit on total output, thinking plus response text."* Under adaptive thinking — on by default on Fable 5, Mythos 5, Opus 5 and Sonnet 5 — the model spends from that budget before it writes a word you will show anyone. Chapter 18 established this; §27.15 prices it.

Its ceiling is per model, not per generation. Fable 5, Opus 5, Sonnet 5, Opus 4.8, 4.7 and 4.6 and Sonnet 4.6 allow **128,000** output tokens; Haiku 4.5, Sonnet 4.5 and Opus 4.5 allow **64,000**. The Batch API raises this to **300,000** with `output-300k-2026-03-24`, on Opus 5, Opus 4.8, Opus 4.7, Opus 4.6, Sonnet 5 and Sonnet 4.6 — **not** Fable 5 and not Haiku 4.5.

And one behaviour that is easy to mistake for a bug: on Claude 4.5 and later, an input plus `max_tokens` that together exceed the context window is **accepted**, and the response stops with `stop_reason: "model_context_window_exceeded"`. Earlier models returned a validation error. Chapter 16 listed that stop reason; this is where it comes from.

27.10 `output_config`: Effort, Task Budget, Format

Three sub-fields, three different kinds of control.

`effort` takes `low`, `medium`, `high`, `xhigh`, `max`. `high` **is the API default** and identical to omitting it. It governs **all** output tokens — text, tool calls and thinking — and is a **behavioural signal, not a cap**; `max_tokens` remains the only hard ceiling. It **does not transfer between models**, so an effort tuned on Sonnet 5 is not the same instruction on Opus 5. **Haiku 4.5 does not support it at all.** `xhigh` is supported on Fable 5, Mythos 5, Opus 5, Opus 4.8, Opus 4.7 and Sonnet 5; `max` additionally on Opus 4.6 and Sonnet 4.6 — and, note carefully, **not** on Opus 5.

`adaptive` is a thinking mode, never an effort value. §27.20 rejects it explicitly because it is the single most common confusion between the two fields.

`task_budget` is an **advisory** total across an agentic loop, behind the beta header `task-budgets-2026-03-13`, on Opus 5, Fable 5, Mythos 5, Opus 4.8 and Opus 4.7 — **not Sonnet 5**. The minimum `total` is **20,000 tokens** and below that the request is a 400. It is not enforced: the countdown is visible only to the model, and an undersized budget produces refusal-like behaviour rather than a truncation.

`format` is Chapter 23's structured outputs, and the one thing to carry forward here is that **changing** `output_config.format` **invalidates the prompt cache for that thread**, so it is a parameter with a cost attached.

27.11 The Remaining Controls

`stop_sequences` ends generation when the model emits one of your strings; `stop_reason` becomes `stop_sequence` and `stop_sequence` names which fired. Chapter 16 covered the branch. Two practical notes: the stop sequence itself is a *string*, not a token, so it interacts with tokenisation (Chapter 6) in ways that make short or partial-token sequences unreliable; and an **empty array** is documented neither as accepted nor as rejected, so $27.20 raises rather than guessing.

`metadata` currently carries one documented field, `user_id`, *"an external identifier for the user who is associated with the request"* which *"should be a uuid, hash value, or other opaque identifier"*, and which *"Anthropic may use to help detect abuse."* The word **opaque** is doing real work: it is a format requirement and a privacy requirement in the same sentence. Any other key in `metadata` is undocumented, and $27.20 raises.

`service_tier` takes `auto` or `standard_only`. Chapter 19 established the rest: **Priority Tier capacity commitments are no longer available for purchase**, and Priority is supported on all models **except Mythos 5, Mythos Preview, Opus 5 and Sonnet 5**.

`speed` selects fast mode — a research preview, **Opus 5 and Opus 4.8 only, not with Batch**, delivering *"up to 2.5× higher output speed"* at $10/$50 per MTok against Opus 5's ordinary $5/$25. Chapter 19 recorded that on 4.7 an unsupported fast-mode request **errors** while on 4.6 it **silently runs at standard speed** — a difference worth a preflight on its own.

`inference_geo` pins inference geography; `"us"` costs **1.1×** on every category, and all values share one rate limit pool.

`stream` is Chapter 17. `fallbacks` is Chapter 25. `context_management`, including the `clear_thinking_20251015` strategy that overrides Chapter 18's per-model thinking preservation, is introduced there and treated fully in Chapter 54. `cache_control` is Chapter 30.

27.12 Visual Analogies

> VISUAL ANALOGY
>
> ### A dimmer switch wired to a room you cannot see
>
> Temperature is a dimmer. Turning it down makes the room darker — but how dark depends on how bright the room already was, and the room changes every token. At $T = 0.5$ one distribution here has an effective vocabulary of 1.1 and another of 20,464.6, from the same switch position.
>
> **Where it breaks.** A dimmer has a monotone, predictable relationship to brightness. Temperature's effect on *behaviour* is not monotone in anything you care about: lowering it makes output more predictable **and** more repetitive **and** sometimes more confidently wrong, and no single number on the dial tells you which you have bought.

VISUAL ANALOGY

A guest list by name versus a guest list by capacity

`top_k` is "the first forty people in the queue" — always forty, whoever they are. `top_p` is "everyone until the room is 90 per cent full" — which is two people when one guest is enormous and forty-two thousand when everybody is small.

Where it breaks. Both metaphors imply the guests exist independently of the door policy. They do not: the truncation renormalises, so the probabilities the survivors end up with are not the ones they arrived with. And unlike a real room, the ordering of the two policies matters — $27.5 measures a total variation distance of 0.0488 between applying them in the two orders.

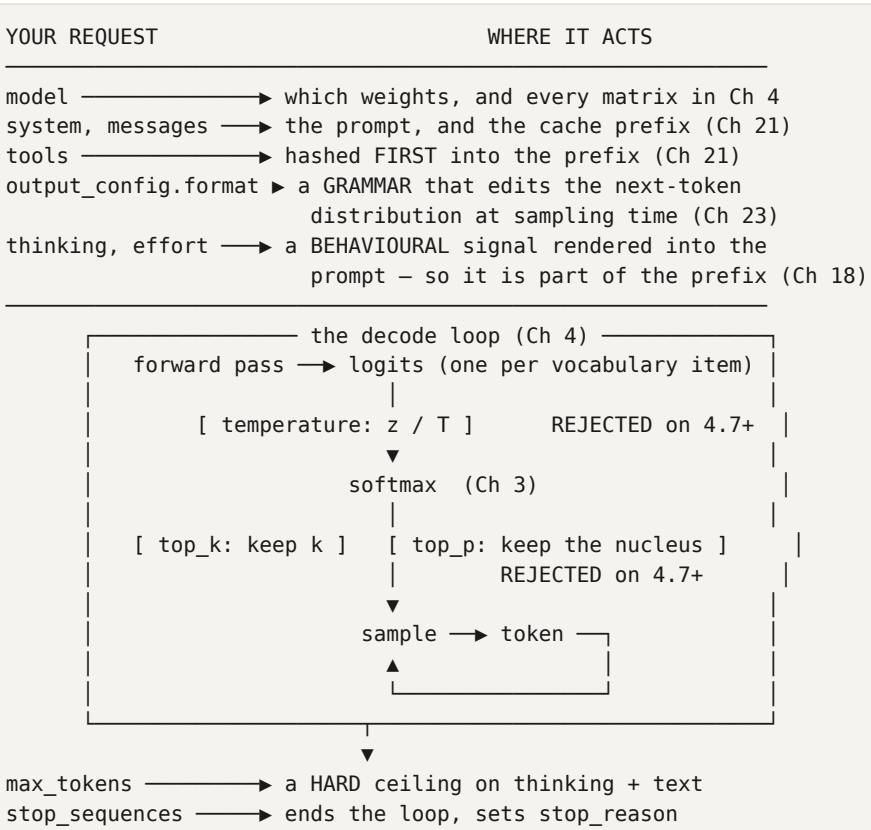
VISUAL ANALOGY

Two accountants adding the same column

Greedy decoding is picking the largest figure in a column two accountants have each totalled. They agree to the penny on every realistic figure — 20,000 trials, zero disagreements. But the numbers are carried to a fixed number of decimal places, so on the rare occasion the top two entries are within a fraction of a penny of each other, which accountant you asked decides the answer.

Where it breaks. Accountants can be asked to re-add. You cannot ask the server to reproduce a summation order, because you never chose it and it depends on how many other people were making requests at the same moment. That is the whole reason reproducibility here is a property of your evaluation design and not of a parameter.

27.13 Architecture: Where a Parameter Takes Effect



```
service_tier, speed, inference_geo ──► which capacity serves it
metadata.user_id ──────► abuse detection, not behaviour
```

Two rules the picture encodes. **The sampling parameters acted inside the loop, on one token at a time**, which is exactly why they were a poor instrument for controlling anything at the level of a document. And `max_tokens` **acts outside the loop, on the whole output**, which is why it is the parameter that silently interacts with thinking.

27.14 The Determinism Measurement

The experiment: compute logits as a matrix product over a hidden dimension of 1,024 into a vocabulary of 2,000, twice — once as a single reduction, once split into two halves and added. In exact arithmetic these are identical. In float32 they are not, and the difference is exactly what a different batch size or kernel produces.

Measurement	Value
trials	20,000
logit discrepancy, mean	**7.11 × 10**
logit discrepancy, 99th percentile	2.38 × 10
logit discrepancy, maximum	**5.01 × 10**
top-two gap, median	**0.1976**
top-two gap, minimum observed	1.07 × 10
top-two gap density near zero	**3.345 per unit**
argmax flips observed	**0**

> NOTE
>
> **A negative result, published**
>
> Zero flips in 20,000 trials. The common claim that floating-point noise routinely scrambles greedy decoding is **not supported by this experiment**. The smallest top-two gap seen was 1.07 × 10 , twice the largest discrepancy — and the median gap is five orders of magnitude larger than the mean discrepancy. On a typical token, greedy decoding is robust with enormous margin.

The estimate that completes the picture uses two measured quantities rather than an assumption. A flip needs the true top-two gap to be smaller than the discrepancy. Near zero the gap's density is very nearly constant — measured at **3.345 per unit** at a threshold of 10^{2}, and 3.400 at 10^{3}, which is the evidence that the linearity holds. So

P(flip per token) ≈ density × mean discrepancy = 3.345 × 7.11 × 10 = 2.377 × 10

— about one token in 420,000. Observing zero flips in 20,000 trials is exactly what that predicts (expected count: 0.048). The estimate and the observation agree.

Now compound it over a generation:

Output length	P(at least one flipped token)
100 tokens	0.024%
1,000 tokens	0.237%
8,000 tokens	1.884%
128,000 tokens	**26.236%**

KEY TAKEAWAYS

The finding in one line

Per token, greedy decoding is deterministic to one part in 420,000. Per 128K-token document, it is a coin flip weighted 1-in-4 against you. No sampling parameter ever changed that, which is the honest reason to stop mourning the one that was removed.

Two caveats stated rather than buried. This is a **model** of the effect, with float32 and a 1,024-dimensional reduction; a real serving stack differs in precision, in reduction width and in kernel. And it measures only the *arithmetic* source of variation. Chapter 14 named a larger one: **pinning fixes weights, not behaviour**, and serving infrastructure — routers, safety classifiers, sampling logic — can change on a stable model ID. Chapter 26 added server-side fallback as another. Floating point is the smallest of the three reasons two identical requests can differ, and it is the only one anybody argues about.

27.15 Examples

Example 1 — the whole correct request on a current model. Note what is absent.

```
msg = client.messages.create(
    model="claude-opus-5",
    max_tokens=16_000,          # thinking AND text
    messages=[{"role": "user", "content": "Summarise the changelog."}],
    output_config={"effort": "medium"},   # the brevity lever
)
```

Example 2 — the migration, in full. Everything on the left is a 400.

```
# BEFORE (Sonnet 4.5, thinking off)
create(model="claude-sonnet-4-5", temperature=0.0, top_p=0.9,
       max_tokens=4000, messages=msgs)
# AFTER (Opus 5)
create(model="claude-opus-5", max_tokens=16_000,
       output_config={"effort": "low", "format": schema},
       messages=msgs)
```

Three substitutions. `temperature=0` for format stability becomes `output_config.format` — a guarantee rather than a hope. `temperature=0` for brevity becomes `effort`. And `max_tokens` **rises**, because it now has to cover thinking.

Example 3 — the one place the old parameters still work. Thinking off, older model:

```
create(model="claude-sonnet-4-5", temperature=0.7,
       thinking={"type": "disabled"}, max_tokens=1024, messages=msgs)
```

Example 4 — and the one narrow window while thinking is on. `top_p` between 0.95 and 1, and nothing else:

```
create(model="claude-sonnet-4-5", top_p=0.98, max_tokens=4096,
       thinking={"type": "enabled", "budget_tokens": 2000},
       messages=msgs)      # temperature or top_k here would be a 400
```

Example 5 — an advisory budget across a loop. Beta header, minimum, and a model that supports it:

```
client.beta.messages.create(
    model="claude-opus-5", max_tokens=32_000,
    output_config={"task_budget": {"total": 40_000}},
    betas=["task-budgets-2026-03-13"], messages=msgs)
```

Example 6 — an opaque user identifier, and only that.

```
create(model="claude-opus-5", max_tokens=1024, messages=msgs,
       metadata={"user_id": hashlib.sha256(
           account_id.encode()).hexdigest()})
```

27.16 Code Walkthrough: Temperature in Four Lines

`sampling.py` implements the softmax exactly as Chapter 3 defined it, with the one adjustment every production implementation makes.

```
def softmax(logits, temperature=1.0):
    z = np.asarray(logits, dtype=np.float64) / float(temperature)
    z = z - z.max()
    e = np.exp(z)
    return e / e.sum()
```

Line 2 is the entire mechanism: temperature is a **divisor on the logits**, applied before anything else. Line 3 subtracts the maximum, which changes nothing mathematically — softmax is invariant to a constant shift — and prevents `exp` from overflowing on a large logit. Lines 4 and 5 are the definition.

```
def zipf_logits(alpha, vocab=VOCAB):
    ranks = np.arange(1, vocab + 1, dtype=np.float64)
    return -alpha * np.log(ranks)
```

Two lines, and they make the rest of the chapter checkable. The softmax of − ·ln(rank) is exactly rank^− / ( ). Dividing by T gives −(/T)·ln(rank), so **sampling at temperature T is sampling from exponent alpha/T** — the closed form asserted in `test_temperature_is_a_divisor_on_the_power_law_exponent`, which compares the two distributions and requires agreement to 10 [12].

```
def top_p_survivors(p, threshold):
    s = np.sort(p)[::-1]
```

```
    c = np.cumsum(s)
    return int(np.searchsorted(c, threshold) + 1)
```

Line 2 sorts descending, line 3 accumulates, line 4 finds the first index where the cumulative mass reaches the threshold and adds one, because the crossing token is **kept**. The whole adaptivity of `top_p` lives in line 3: the cumulative sum depends on the distribution's shape, so the answer does too. Contrast the entire body of `top_k_survivors`, which is `min(k, nonzero)` and cannot depend on shape at all.

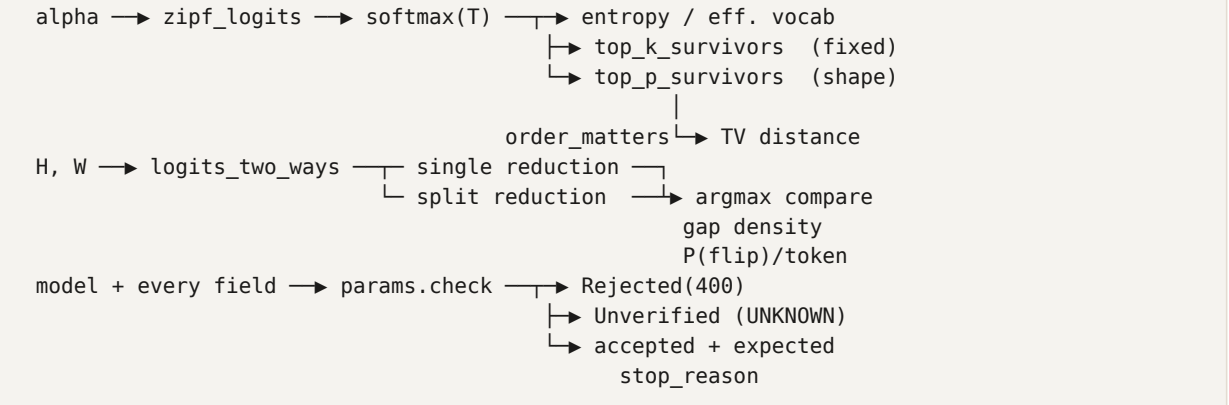
```
def logits_two_ways(H, W):
    a = (H @ W).astype(np.float32)
    half = W.shape[0] // 2
    b = ((H[..., :half] @ W[:half])
         + (H[..., half:] @ W[half:])).astype(np.float32)
    return a, b
```

Line 2 computes the logits as one reduction over the hidden dimension. Lines 4 and 5 compute the *same mathematical quantity* as two half-reductions added together. There is no approximation and no noise injected anywhere: the only difference is the order in which floating-point numbers are added, and that is sufficient. Everything in §27.14 comes from comparing these two arrays.

27.17 Mini Project: The Parameter Bench

What it is. A sampling laboratory and a parameter preflight. The first makes the removed controls concrete enough to argue about; the second refuses, offline and with no key, everything the documentation says is refused.

```
ch27/
  wire.py         copied from ch16/ — the request body
  registry.py     copied from ch14/ — model IDs with provenance
  sampling.py     softmax, temperature, top_k, top_p, ordering
  determinism.py  the two-reduction-order experiment
  params.py       23 rejections, 6 acceptances, 4 UNKNOWNs
  report.py       prints every figure in this chapter
  test_ch27.py    30 tests
```




```
cd ch27 && python3 report.py         # every figure below
cd ch27 && python3 -m pytest -q      # 30 passed
```

Deployment. `params.check` composes with Chapter 26's `preflight`: that one validates the headers and the credential, this one validates the body. Run both in your outbound middleware and a whole class of 400 disappears from production before it is ever sent.

27.18 Panel A — Temperature, Measured

Six temperatures against two distributions, 50,000 tokens each. Effective vocabulary is 2^entropy: the number of equally likely tokens the distribution behaves like.

T	peaked top-1	peaked eff. vocab	flat top-1	flat eff. vocab
0.1	1.0000	**1.0**	0.8875	1.6
0.5	0.9830	**1.1**	0.0121	**20,464.6**
0.7	0.9389	1.3	0.0022	37,365.8
1.0	0.8319	2.0	0.0006	44,975.5
1.5	0.6079	5.1	0.0002	48,117.4
2.0	0.3841	**23.5**	0.0001	49,026.6

Two readings. Down the columns: temperature works, monotonically and dramatically — on the peaked distribution it moves the effective vocabulary from 1.0 to 23.5, a factor of 23. Across the rows: **the same setting means different things**. At $T = 0.5$ the peaked distribution is decided and the flat one is choosing among twenty thousand.

That is not an argument that temperature was useless. It is an argument that it was a **global** control on a **local** quantity, which is a specific kind of unsatisfactory.

27.19 Panel B — Truncation, and Why One of These Ports

Setting	Value	Peaked	Flat	Ratio
`top_p`	0.5	1	17,228	17,228×
`top_p`	**0.9**	**2**	**42,523**	**21,261×**
`top_p`	0.95	3	46,209	15,403×
`top_p`	0.99	6	49,234	8,206×
`top_k`	1	1	1	1×
`top_k`	5	5	5	1×
`top_k`	40	40	40	1×
`top_k`	100	100	100	1×

And the ordering result, which says these two parameters are not even independent:

Distribution	TV distance, T-then-p vs p-then-T	survivors
peaked	**0.0488**	1 vs 2
flat	0.0238	40,530 vs 42,523

Three things follow. **A `top_p` value is not a portable configuration** — it means 2 tokens on one prompt and 42,523 on the next. **A `top_k` value is portable and frequently wrong**, admitting 39 tokens the model had ruled out. And **"temperature 0.7, top_p 0.9" is not one configuration**, because two implementations applying them in different orders produce measurably different distributions.

Chapter 15's rule applies to sampling parameters exactly as it applied to models: **the evaluation set is the evidence**. There was never a defensible way to pick these numbers by reasoning about them.

27.20 Panel C — Zero Flips in Twenty Thousand

Measurement	Value
trials	20,000
logit discrepancy, mean	7.11 × 10
logit discrepancy, 99th percentile	2.38 × 10
logit discrepancy, maximum	5.01 × 10
top-two gap, median	0.1976
top-two gap, minimum observed	1.07 × 10
gap density near zero	3.345 per unit
P(flip) per token, estimated	**2.377 × 10**
argmax flips observed	**0**

The negative result first: **zero flips in 20,000 trials.** The folklore is not supported. The median top-two gap is 0.1976 against a mean discrepancy of 7.11 × 10 — a margin of about **278,000×**.

The estimate second, and it is a product of two measured numbers rather than an assumption. A flip requires the true gap to fall below the discrepancy; near zero the gap density is measured at 3.345 per unit (and 3.400 at a ten-times-tighter threshold, which is the evidence the linearity holds). So P(flip) per token = 3.345 × 7.11 × 10 = **2.377 × 10** . Over 20,000 trials that predicts **0.048** flips, and zero were observed. The estimate and the observation are consistent, which is the only reason to trust either.

Compounded over a generation:

Output length	P(at least one flipped token)
100	0.024%
1,000	0.237%

Output length	P(at least one flipped token)
8,000	1.884%
128,000	**26.236%**

KEY TAKEAWAYS

The finding in one line

One token in 420,000; one document in four. Reproducibility was never a property of a parameter and is not a property of the request. It is a property of your evaluation design, which is what Chapters 24 and 25 were for.

27.21 Panel D — `max_tokens` Against Thinking

A workload whose *answer* needs about 1,200 tokens, running on a model with adaptive thinking on by default. Twenty thousand simulated requests with a log-normal thinking distribution — median **3,025** tokens, p90 **8,381**, p99 **19,395**.

`max_tokens`	requests truncated
2,000	**94.92%**
4,000	**53.89%**
8,000	15.27%
16,000	2.30%
32,000	0.19%

A cap of 4,000 — more than three times the answer length, and generous by pre-thinking instincts — truncates more than **half** of all requests. The failure is not a 400; it is `stop_reason: "max_tokens"` and an answer that stops mid-sentence, and Chapter 16 recorded that `content[0].text` silently returns a truncated string for exactly this case.

Two corollaries. **Size `max_tokens` from the thinking distribution, not the answer length** — and measure the distribution rather than guessing it, because `usage.output_tokens_details.thinking_tokens` reports the split. And `max_tokens` **is a ceiling, not a bill**: raising it from 4,000 to 16,000 costs nothing extra on requests that do not use it, and the only reason not to is the SDK guard Chapter 26 noted, which requires streaming above 21,333.

27.22 Panel E — Twenty-Three Rejections, Six Acceptances, Four Unknowns

#	Case	Prov.	Status
1	`temperature=0` on Opus 5	DOC	400

#	Case	Prov.	Status
2	`temperature=0.7` on Sonnet 5	DOC	400
3	`top_p=0.9` on Fable 5	DOC	400
4	`top_k=40` on Opus 4.7	DOC	400
5	`top_p` on Mythos Preview	DOC	400
6	`temperature` on Sonnet 4.5 **while thinking**	DOC	400
7	`top_k` on Sonnet 4.5 while thinking	DOC	400
8	`top_p=0.5` on Sonnet 4.5 while thinking	DOC	400
9	`max_tokens` of 0	DOC	400
10	`max_tokens` above the model ceiling	DOC	400
11	300K output in a batch without the beta	BUILT	400
12	`effort` on Haiku 4.5	DOC	400
13	`effort: xhigh` on Sonnet 4.6	DOC	400
14	`effort: max` on Opus 5	DOC	400
15	`effort: adaptive` (a thinking mode, not an effort)	BUILT	400
16	`task_budget` without its beta header	DOC	400
17	`task_budget` on Sonnet 5	DOC	400
18	`task_budget` below 20,000	DOC	400
19	prefill on Opus 5	DOC	400
20	prefill on Sonnet 4.6	DOC	400
21	`service_tier: "priority"`	BUILT	400
22	fast mode on Sonnet 5	DOC	400
23	fast mode with Batch	DOC	400

Rejected 23 of 23, offline, with no key.

Six requests it accepts, each chosen because a plausible reading of the rules would reject it:

Case	Why it is accepted
Opus 5 with only the required fields	the correct shape on a current model
`top_p=0.98` on Sonnet 4.5 while thinking	inside the documented 0.95–1 window
`temperature=0.7` on Sonnet 4.5 with thinking **disabled**	the restriction is conditional on older models
`effort: xhigh` on Opus 5	supported

Case	Why it is accepted
`effort: max` on Opus 4.6	`max` is supported there and **not** on Opus 5
300K output on Opus 5 in a batch **with** `output-300k-2026-03-24`	the documented exception

And four cases the documentation covers neither way, which raise:

Case	Why UNKNOWN
an empty `stop_sequences` array	neither accepted nor rejected in the documentation
a `metadata` key other than `user_id`	only `user_id` is documented
a model this book has not verified	Chapter 14's rule
128K output on Mythos Preview	its output ceiling was not verified on 1 August 2026

COMMON MISTAKE

The parameter that type-checks and 400s

Symptom. A 400 on a request whose types are correct and whose CI is green. **Cause.** *"The SDK request types still define these fields for compatibility with earlier models."* A type is not a contract with the server. **Fix.** Delete the field. Do not set it to a default; omit it. **Detection.** A preflight over the model matrix — 23 of 23 above, all offline.

27.23 Evaluation and Deployment

Measure	Value
Documented rejections reproduced offline	23 of 23
Documented acceptances confirmed	6 of 6
UNKNOWN cases that raise	4 of 4
Closed forms asserted against the implementation	2 (temperature/exponent; gap exposure)
Determinism trials	20,000, zero flips
Tests	30, passing, no key, no network

Deploy `params.check` next to Chapter 26's `preflight`. Between them they cover the credential, the headers, the betas, the endpoint, the size and every body field this book has verified — and the whole thing runs in microseconds without a network.

27.24 Hands-on Labs

Lab 27.1 — Make temperature concrete. Build a power-law distribution, sweep the temperature and plot effective vocabulary. *Success criterion:* you can state, without running anything, what temperature 0.5 does to a distribution with = 3 versus = 0.35, and you are within 10 per cent.

Lab 27.2 — Show that `top_p` does not port. Compute survivor counts for a fixed `top_p` on two distributions of different shape. *Success criterion:* your ratio exceeds 1,000×, and you can explain to a colleague why `top_k` gives 1×.

Lab 27.3 — Break associativity. Compute one matrix product two ways and measure the maximum absolute disagreement. *Success criterion:* the disagreement is strictly greater than zero and smaller than 10 , and you can say why splitting at an aligned boundary can give exactly zero.

Lab 27.4 — Size `max_tokens` honestly. Take a real thinking-token distribution from `usage.output_tokens_details.thinking_tokens`, or simulate one, and compute the truncation rate at four caps. *Success criterion:* you can state the cap that keeps truncation below one per cent.

Lab 27.5 — Preflight your own call sites. Run every model-and-parameter combination your codebase produces through a checker. *Success criterion:* it rejects at least one thing you did not know was rejected, or you can prove there was nothing to find.

27.25 Lab Solutions

27.1. Effective vocabulary is 2^H where H = − p log p. For a power law, temperature T is a divisor on , so = 3 at T = 0.5 behaves like = 6 — effectively decided, eff. vocab 1.1 — while = 0.35 at T = 0.5 behaves like = 0.7, which over 50,000 tokens is still enormously diffuse at 20,464.6. See `sampling.temperature_sweep`.

27.2. Sort descending, take the cumulative sum, and find the first index reaching the threshold. The peaked case gives 2 at `top_p = 0.9` and the flat case 42,523, a ratio of 21,261×. `top_k` returns k by construction. See `sampling.top_p_survivors`.

27.3. `H @ W` versus `H[:, :h] @ W[:h] + H[:, h:] @ W[h:]`. The mean disagreement here is 7.11 × 10 and the maximum 5.01 × 10 . If you get exactly zero, your split aligned with the library's own pairwise-summation tree so both paths performed identical additions — split at an odd offset, or use the BLAS path as `logits_two_ways` does. That is itself worth knowing: **the effect depends on the implementation, not only on the mathematics.**

27.4. Truncation happens when thinking + answer > cap. At an answer of 1,200 and the distribution in Panel D, a cap of 16,000 gives 2.30 per cent and 32,000 gives 0.19 per cent. Note that raising the cap costs nothing on requests that do not use it. See `params.budget_table`.

27.5. The checker is a matrix, not a list: the same parameter is legal on one model and a 400 on another, and on the older generation legality depends on whether thinking is on. Order the checks so the model-existence test comes first and raises UNKNOWN, because a checker that silently passes an unknown model is worse than no checker. See `params.check`.

27.26 Best Practices

BEST PRACTICE

Omit the sampling parameters entirely

Not zero, not the default — absent. They are rejected on Claude 4.7 and later, and on earlier models omitting them is also correct. Chapter 18's rule applies: setting a parameter to its default is equivalent to omitting it, and both are part of the cache prefix.

BEST PRACTICE

Reach for the replacement that matches the intent

Temperature was doing one of three jobs. For **format**, use `output_config.format` (Chapter 23) — a guarantee, not a hope. For **length or thoroughness**, use `effort`. For **behaviour**, use the prompt, which is what the documentation says: *"use prompting to guide the model's behavior instead."*

BEST PRACTICE

Size `max_tokens` from the thinking distribution

Measure `usage.output_tokens_details.thinking_tokens` on your own workload and set the cap so truncation is under one per cent. A cap sized for the answer alone truncated 53.89 per cent of requests in Panel D.

BEST PRACTICE

Raise `max_tokens` freely; it is a ceiling, not a bill

You are billed for tokens produced, not for the cap. The only constraints are the per-model ceiling and the SDK's streaming requirement above 21,333.

BEST PRACTICE

Read the per-model matrix, never the generation

`effort: max` is supported on Opus 4.6 and **not** on Opus 5. `task_budget` works on Opus 5 and **not** on Sonnet 5. Chapter 14's rule stands: capability does not travel in a bundle.

BEST PRACTICE

Never claim reproducibility from a parameter

Assert distributions, not strings. A regression suite that requires byte-identical output will eventually fail on a 128K generation for reasons no one can reproduce — 26.2 per cent of the time, by Panel C.

BEST PRACTICE

Send an opaque `user_id` and nothing else

A uuid or a hash. It helps abuse detection, and the word *opaque* in the documentation is a privacy requirement as much as a format one.

BEST PRACTICE

Preflight the body next to the headers

Chapter 26 validated everything around the request; this chapter validates the request. Compose them in one middleware and the 400s stop leaving your process.

BEST PRACTICE

Treat `effort` as untransferable

It is a behavioural signal, not a cap, and it does not mean the same thing on two models. Re-tune it after a migration, and remember Chapter 15's finding that a `(model, effort)` pair is the real unit of choice.

BEST PRACTICE

Budget for `stop_reason: "max_tokens"` as a normal outcome

It is not an error, it produces an HTTP 200, and `content[0].text` will hand you a truncated string without complaint. Branch on `stop_reason` first, always.

27.27 Common Mistakes

COMMON MISTAKE

Carrying `temperature=0` through a migration

Symptom. Every request 400s after a one-line model change; the type checker was green. **Cause.** The SDK types keep the field for older models; the API rejects it. **Fix.** Remove it. Replace the *intent* — format, brevity or behaviour — with the matching modern control. **Detection.** `params.check`, cases 1–5.

COMMON MISTAKE

Believing `temperature=0` gave reproducibility

Symptom. A regression suite asserting string equality fails intermittently and irreproducibly, usually on the longest outputs. **Cause.** Greedy decoding is argmax over floating-point sums whose accumulation order you do not control. **Fix.** Assert on distributions, graders and intervals (Chapter 24), not on bytes. **Detection.** Panel C: 26.236 per cent of 128K generations contain at least one flippable token.

COMMON MISTAKE

Porting a `top_p` between workloads

Symptom. A carefully tuned `top_p` behaves unrecognisably on a new prompt family. **Cause.** `top_p` truncates by mass, so its candidate set varies by a factor of 21,261 across the two shapes in Panel B. **Fix.** Re-tune per workload against an evaluation set — or, on current models, note that the parameter no longer exists and stop. **Detection.** Compute survivor counts on a sample of real distributions.

COMMON MISTAKE

Sizing `max_tokens` for the answer

Symptom. Answers stop mid-sentence on a model where they never used to. **Cause.** `max_tokens` is a hard limit on **thinking plus text**, and adaptive thinking is on by default on current models. **Fix.** Size from the measured thinking distribution. **Detection.** Alert on the rate of `stop_reason: "max_tokens"`; in Panel D it was 53.89 per cent at a cap of 4,000.

COMMON MISTAKE

Passing `adaptive` as an effort value

Symptom. A 400 on a request that looks correct. **Cause.** `adaptive` is a **thinking mode**; `effort` takes `low`/`medium`/`high`/`xhigh`/`max`. The two fields are adjacent and confusable. **Fix.** `thinking={"type": "adaptive"}`, `output_config={"effort": ...}`. **Detection.** `params.check`, case 15.

COMMON MISTAKE

Assuming `max` effort is the top of a ladder every model has

Symptom. A 400 for `effort: max` on Opus 5. **Cause.** `max` is supported on Opus 4.6 and Sonnet 4.6, not on the 5 generation, where `xhigh` is the ceiling. **Fix.** Read the matrix. **Detection.** `params.check`, cases 13 and 14.

COMMON MISTAKE

Setting `task_budget` and expecting it to be enforced

Symptom. A run exceeds its budget with no error. **Cause.** `task_budget` is **advisory**; the countdown is visible only to the model. **Fix.** Use `max_tokens` for a hard ceiling and cost controls (Chapter 30) for a hard budget. **Detection.** Compare `usage` against the budget across a sample; the gap is the point.

COMMON MISTAKE

Putting an identifier in `metadata.user_id`

Symptom. A privacy review finding, or a data-retention question you cannot answer. **Cause.** The field is documented as *"a uuid, hash value, or other opaque identifier"* and an email address is none of those. **Fix.** Hash it. Keep the mapping on your side. **Detection.** A lint over the call site, or the leak scanner from Chapter 26 extended to obvious identifiers.

COMMON MISTAKE

Requesting fast mode on the wrong model

Symptom. Either a 400 or — worse — no error and no speed-up. **Cause.** Fast mode is Opus 5 and Opus 4.8 only. Chapter 19 recorded that 4.7 **errors** and 4.6 **silently runs at standard speed**. **Fix.** Preflight it. Case 22. **Detection.** Compare measured output rate against the tier you believe you are on.

COMMON MISTAKE

Treating a context-window overflow as a validation error

Symptom. A response with no error and a stop reason your handler has no arm for. **Cause.** On Claude 4.5 and later, input plus `max_tokens` exceeding the window is **accepted** and returns `stop_reason: "model_context_window_exceeded"`. **Fix.** Handle the stop reason. Chapter 16 listed it; `params.check` predicts it. **Detection.** `test_context_overflow_is_accepted_and_reported_as_a_stop_reason`.

27.28 Performance Considerations

No parameter in this chapter is free of latency implications, and only one of them is about latency. `effort` governs how many tokens are produced, and Chapter 11's finding stands: **decode dominates wall clock, and generating fewer tokens is the only lever that improves latency, throughput and cost together.** Lowering effort is therefore the highest-leverage change available in the body of the request.

`max_tokens` **costs nothing until it is used.** Raising a cap does not raise a bill. What it does raise is the *variance* of your latency, because the model is now permitted to think for longer — and the p99 of a workload with a 32,000-token cap is not the p99 of the same workload at 4,000.

Every parameter that is part of the prompt is part of the cache prefix. Chapter 18 established that the thinking configuration and the resolved effort level are rendered into the prompt. So changing `effort` between requests invalidates the prefix and costs a full re-prefill, and Chapter 20 priced what that is worth. A per-request effort tuner is not free; Chapter 18 measured one costing **133.3 per cent** more than a single fixed configuration.

`speed` **and** `inference_geo` **are the only fields that change which capacity serves you.** Fast mode is up to 2.5× output speed at double the price; `inference_geo: "us"` is 1.1× on every category for no speed change at all.

The sampling parameters had no measurable latency cost, which is part of why their removal is a behavioural change rather than a performance one.

27.29 Security Considerations

`metadata.user_id` **is a data-flow decision.** You are sending an identifier to a third party for abuse detection. The documented requirement — uuid, hash, or other opaque identifier — is the mitigation, and a hash you compute yourself keeps the mapping inside your boundary.

A `stop_sequence` **is a string an attacker may be able to emit.** If your stop sequence appears in user-controlled content that is echoed into the output, generation can be terminated early by the input. Choose sequences that cannot occur naturally, and treat a `stop_sequence` stop reason as a signal to inspect rather than trust.

`max_tokens` **is a denial-of-wallet control.** It is the only hard ceiling on output, and output is five times the price of input on every current model. An unbounded cap on a user-triggered path is a cost incident waiting for a prompt that makes the model verbose.

`task_budget` **is not a security control.** It is advisory. Anything you need enforced belongs in `max_tokens` or in the spend caps of Chapter 19.

Parameters are part of the cached prefix, and caches are scoped. Chapter 26 recorded that prompt caches are per workspace on Anthropic-operated platforms and per organisation on Bedrock and Google Cloud. A configuration difference between tenants is therefore also a cache-partitioning decision.

27.30 Real Production Examples

A research team's green build and red production. The chapter's opening. `temperature=0` survived a model migration because it type-checked. The deeper finding was that their reproducibility assumption had never held; the fix was a grader-based regression suite rather than string equality.

A documentation pipeline that started truncating. After enabling a current model, roughly half of a summarisation workload began returning `stop_reason: "max_tokens"` with no configuration change. `max_tokens` had been 4,000 for years, sized for a 1,200-token answer. Adaptive thinking was spending the rest. They measured `thinking_tokens`, moved the cap to 16,000, and truncation fell to about two per cent at no additional cost.

A vendor whose `top_p` "stopped working". A `top_p` of 0.9 tuned on a classification workload was reused for open-ended drafting and produced near-random output. Nothing had broken: the nucleus had gone from a couple of tokens to tens of thousands, exactly as Panel B predicts.

An agent platform that budgeted with `task_budget`. Runs regularly exceeded the budget. The parameter is advisory and always was; the hard ceiling they needed was `max_tokens` per call plus a workspace spend limit.

27.31 Certification Notes

Concept	Domain	Weight	What is examinable
Required body fields	D4 Prompt Engineering & Structured Output	20%	`model`, `max_tokens`, `messages`
Sampling parameters removed	D4	20%	Claude 4.7+ and Mythos Preview; non-default is a 400
The legacy restriction	D4	20%	older models: only while thinking is on; `top_p` 0.95–1
The documented replacement	D4	20%	prompting; structured outputs; effort
`max_tokens` semantics	D5 Context Management & Reliability	15%	hard limit on **thinking plus text**; per-model ceiling
The 300K batch exception	D5	15%	`output-300k-2026-03-24`, and which models

Concept	Domain	Weight	What is examinable
Context overflow	D5	15%	accepted on 4.5+, `model_context_window_exceeded`
`effort`	D1 Agentic Architecture & Orchestration	27%	five values, `high` default, no Haiku, `max` not on Opus 5
`task_budget`	D1	27%	advisory, 20,000 minimum, beta header, not Sonnet 5
`metadata.user_id`	D5	15%	opaque identifier; abuse detection
`service_tier` and `speed`	D5	15%	`auto`/`standard_only`; fast mode Opus 5 and 4.8 only

CERTIFICATION NOTE

Exam tips

The single highest-yield fact: `temperature`, `top_p` **and** `top_k` **return a 400 on Claude 4.7 and later, and on Mythos Preview.** If an option says "temperature 0 is still allowed for determinism", it is wrong — **there is no exemption**.

On **older** models the restriction is conditional: only while thinking is on, `temperature` and `top_k` are incompatible, and `top_p` is allowed **0.95 to 1**. That window is a favourite distractor.

`max_tokens` **is a hard limit on thinking plus text.** Any question where a current model truncates unexpectedly is asking about this.

Distinguish the two adjacent fields: `adaptive` **is a thinking mode;** `effort` **takes low/medium/high/xhigh/max, and** `high` **is the default.** `max` is on Opus 4.6 and Sonnet 4.6, **not** on Opus 5.

`task_budget` **is advisory**, needs `task-budgets-2026-03-13`, has a **20,000** minimum, and is **not** on Sonnet 5.

And if a question asks how to get deterministic output, the answer is that you cannot, and the correct engineering response is an evaluation suite.

27.32 Twenty Practice Questions

1. Which three fields are required in a Messages request?
2. Which models reject a non-default `temperature`, and with what status?
3. What does the documentation offer as the replacement for the sampling parameters?
4. Why does setting `temperature` on Opus 5 type-check in Python?
5. On Sonnet 4.5 with thinking enabled, which sampling parameters are usable and in what range?
6. What does temperature do to a power-law next-token distribution?
7. Define effective vocabulary and say what it is for a uniform distribution over 64 tokens.
8. How many tokens survive `top_p = 0.9` on a peaked versus a flat distribution in this chapter's measurement?

9. How many survive `top_k = 40` on each?
10. Does applying temperature before top-p give the same result as the reverse?
11. Is greedy decoding deterministic? Justify the answer numerically.
12. What is the estimated probability that a single token flips under a different reduction order, and what is it over 128,000 tokens?
13. What does `max_tokens` bound?
14. What are the output ceilings for Opus 5 and Haiku 4.5?
15. How do you obtain a 300,000-token output, and on which models?
16. What happens when input plus `max_tokens` exceeds the context window on a 4.5-or-later model?
17. List the five `effort` values, name the default, and name a model that supports none of them.
18. Which models support `task_budget`, what is the minimum, and is it enforced?
19. What belongs in `metadata.user_id`?
20. On which models is fast mode available, and what happens if you request it on Opus 4.6?

27.33 Answers and Explanations

1. `model`, `max_tokens`, `messages`. `system` is a top-level field but optional; naming it as required is the standard distractor.
2. **Claude 4.7 and later, plus Mythos Preview; HTTP 400.** Answers naming only Opus 5, or only the 5 generation, are too narrow.
3. **Prompting** — *"use prompting to guide the model's behavior instead"* — supplemented by structured outputs for format and `effort` for length. There is no replacement parameter.
4. **The SDK request types still define the fields for compatibility with earlier models.** The rejection is server-side; a type is not a contract with the server.
5. `top_p` **only, and only between 0.95 and 1.** `temperature` and `top_k` are incompatible with thinking on those models. With thinking **disabled**, all three are usable.
6. **It divides the exponent.** Logits of $-s \cdot \ln(\text{rank})$ at temperature T give a power law with exponent s/T.
7. **2 raised to the entropy in bits** — the number of equally likely tokens the distribution behaves like. For a uniform distribution over 64 tokens it is exactly **64**.
8. **2 and 42,523**, a ratio of 21,261×.
9. **40 and 40.** `top_k` is a fixed cut by construction.
10. **No.** The measured total variation distance is 0.0488 on the peaked distribution, with survivor counts of 1 versus 2.
11. **In exact arithmetic yes; in practice no.** Zero flips were observed in 20,000 trials, but the estimated per-token flip probability is 2.377×10 , which compounds.
12. **2.377×10 per token; 26.236 per cent over 128,000 tokens.**
13. **Total output — thinking plus response text.** "Response text only" is the mistake this chapter is built around.
14. **128,000 and 64,000.**

15. **The `output-300k-2026-03-24` beta on the Batch API**, on Opus 5, Opus 4.8, Opus 4.7, Opus 4.6, Sonnet 5 and Sonnet 4.6 — not Fable 5, not Haiku 4.5.
16. **The request is accepted** and the response stops with `stop_reason: "model_context_window_exceeded"`. Earlier models returned a validation error.
17. **`low`, `medium`, `high`, `xhigh`, `max`; `high` is the default; Haiku 4.5 supports none.**
18. **Opus 5, Fable 5, Mythos 5, Opus 4.8 and Opus 4.7 — not Sonnet 5. Minimum 20,000 tokens. It is advisory, not enforced.**
19. **An opaque identifier — a uuid or hash.** Anthropic may use it to help detect abuse. Not an email address or a name.
20. **Opus 5 and Opus 4.8 only.** On Opus 4.6 the request **silently runs at standard speed**; on 4.7 it errors — a documented asymmetry worth remembering.

27.34 Ten Practical Exercises

1. Grep your codebase for `temperature`, `top_p` and `top_k` and delete every occurrence on a current model.
2. For each deletion, write down which of the three jobs it was doing, and which modern control replaces it.
3. Measure `usage.output_tokens_details.thinking_tokens` across a week of production traffic and plot the distribution.
4. Compute the `max_tokens` that keeps your truncation rate under one per cent, and set it.
5. Add a `stop_reason: "max_tokens"` counter to your metrics if you do not have one.
6. Run one prompt twenty times on a current model and measure how often the output is byte-identical.
7. Replace one string-equality assertion in your regression suite with a grader from Chapter 24.
8. Check every `effort` value in your codebase against the per-model matrix.
9. Confirm that whatever you put in `metadata.user_id` is opaque.
10. Run `params.check` over every model-and-parameter combination your code can produce.

27.35 Five Challenge Labs

1. **The sampling museum.** Implement temperature, top-k, top-p and min-p from scratch, and produce the plot that shows which of them keep the candidate-set size stable across distribution shapes.
2. **The reproducibility budget.** Given your output-length distribution, compute the probability that a batch of N generations contains at least one reduction-order flip, and decide what your regression suite should assert as a consequence.
3. **The truncation guard.** Build a middleware that predicts, from the prompt and the historical thinking distribution, the probability that this request will hit `max_tokens`, and raises the cap automatically when it exceeds a threshold.
4. **The effort tuner that pays for itself.** Chapter 18 measured a per-request effort tuner costing 133.3 per cent more than one fixed configuration because it kept invalidating the cache. Build one that

batches requests by effort level so the prefix is shared, and measure whether it wins.

5. **The parameter differ.** Extend Chapter 14's migration differ to the body: given two `(model, parameters)` pairs, enumerate every field that becomes invalid, every default that changes meaning, and every ceiling that moves.

27.36 Ten Interview Questions

1. Explain what temperature does, in terms of the softmax.
2. Why is `top_p` harder to port between workloads than `top_k`?
3. A colleague says they set `temperature=0` for reproducible output. What do you tell them?
4. What replaced the sampling parameters, and is that a good trade?
5. Why did the type checker not catch a `temperature` on Opus 5?
6. A summarisation service starts truncating after a model upgrade with no config change. Diagnose it.
7. What is the difference between `effort` and `max_tokens`?
8. What is `task_budget` for, and what is it not for?
9. How would you design a regression suite for a system whose output is not reproducible?
10. Which request parameters affect the prompt cache, and why does that matter?

27.37 Summary

The Messages API asks for three fields and offers about fifteen more. This chapter took them one at a time, and the through-line turned out to be a single question: **what is this parameter actually a control on?**

The sampling parameters were controls on the shape of one token's probability distribution. Temperature divides the logits before the softmax, which on a power-law distribution is exactly a divisor on the exponent — an unusually clean mechanism that makes the whole thing checkable. Top-k keeps a fixed number of candidates; top-p keeps a fixed amount of probability mass. Those are different in a way that matters enormously in practice: at `top_p = 0.9`, a confident distribution keeps **2** tokens and an open-ended one keeps **42,523**, a factor of **21,261** from the same setting, while `top_k = 40` keeps exactly forty in both. A parameter whose meaning changes by four orders of magnitude with the prompt is not a configuration; it is a variable. And the two do not even commute — the same "temperature 0.7, top_p 0.9" applied in the two orders differs by a total variation distance of **0.0488**.

All three are now rejected. On Claude 4.7 and later, and on Mythos Preview, a non-default value is a 400, and the SDK will not warn you because the request types keep the fields for older models — *"code that sets them type-checks, but the API rejects the request server-side."* That is the sixth member of the family Chapter 23 opened and Chapter 26 catalogued: a convenience that is not the contract. The documented replacement is not another parameter but prompting, with structured outputs for format and `effort` for length, and on the evidence of the last three chapters that is a defensible trade: two real losses in generative diversity against two illusory ones.

The most durable of the illusions was determinism, and it deserved a measurement rather than an assertion. Greedy decoding is argmax over dot products, and floating-point addition is not associative, so

a different summation order — a different batch size, a different kernel — can flip the answer. Across **20,000 trials this chapter observed zero flips**, and that negative result is worth publishing: the folklore overstates the effect badly, with a median top-two gap of 0.1976 against a mean discrepancy of 7.11×10 . But the same measurements give a per-token flip probability of **2.377×10** , and over a 128,000-token generation that compounds to **26.2 per cent**. One token in 420,000; one long document in four. `temperature = 0` never fixed that, and the correct engineering response was always the one Chapter 24 built.

The rest is bookkeeping with teeth. `max_tokens` **is a hard ceiling on thinking plus text**, so a cap of 4,000 sized for a 1,200-token answer truncates **53.89 per cent** of requests once adaptive thinking shares it — a failure that arrives as an HTTP 200 and a sentence that stops. `effort` is a behavioural signal with five values, a default of `high`, no support at all on Haiku 4.5, and a `max` level that exists on Opus 4.6 and not on Opus 5. `task_budget` is **advisory**, needs a beta header and a 20,000-token minimum, and is absent from Sonnet 5. And the preflight that encodes all of it reproduced **23 of 23** documented rejections, confirmed **6 of 6** documented acceptances, and raised on **4 of 4** cases the documentation covers neither way — offline, in microseconds, with no key.

KEY TAKEAWAYS

1. **Three fields are required**: `model`, `max_tokens`, `messages`. Everything else is a decision, including the ones you make by omission.
2. **Temperature is a divisor on the logits** — and on a power-law distribution, exactly a divisor on the exponent.
3. `top_k` **is a fixed cut;** `top_p` **is not.** The same `top_p = 0.9` kept 2 tokens and 42,523 in this chapter's measurement.
4. **Temperature and top-p do not commute**, so "the same settings" in two libraries were never the same experiment.
5. **All three are rejected on Claude 4.7 and later and on Mythos Preview**, with no exemption for `temperature = 0`.
6. **The SDK types are not the contract.** They keep the fields for older models; the API rejects them server-side.
7. **On earlier models the restriction is conditional** on thinking being on, and `top_p` survives in a 0.95–1 window.
8. `temperature = 0` **never gave reproducibility.** Zero flips in 20,000 trials, 2.377×10 per token, **26.2 per cent** over a 128K generation.
9. `max_tokens` **bounds thinking plus text**, and a cap sized for the answer truncated 53.89 per cent of requests.
10. **Read the per-model matrix.** `effort: max` is on Opus 4.6 and not Opus 5; `task_budget` is on Opus 5 and not Sonnet 5.

27.38 Glossary

Term	Definition
Adaptive thinking	A **thinking mode**, not an effort value; on by default on Fable 5, Mythos 5, Opus 5 and Sonnet 5.
Effective vocabulary	2 raised to the entropy in bits: the number of equally likely tokens a distribution behaves like.
Effort	`output_config.effort`, a behavioural signal over all output tokens; five values, default `high`.
Greedy decoding	Always selecting the highest-probability token; deterministic in exact arithmetic only.
Logit	The unnormalised score a model assigns a token before the softmax.
`max_tokens`	A required hard ceiling on **total** output — thinking plus response text.
`metadata.user_id`	An opaque external user identifier that Anthropic may use to help detect abuse.
Nucleus sampling	`top_p`: keep the smallest set of tokens whose probability sums to at least p.
Power-law logits	Logits of the form $-s \cdot \ln(\text{rank})$, whose softmax is an exact power law in rank.
Sampling parameters	`temperature`, `top_p`, `top_k` — rejected with a 400 on Claude 4.7 and later.
Softmax	The map from logits to probabilities, $p \quad \exp(z)$.
`stop_sequences`	Strings that end generation, setting `stop_reason` to `stop_sequence`.
Task budget	`output_config.task_budget`, an **advisory** total across an agentic loop; 20,000-token minimum.
Temperature	A divisor on the logits before the softmax; a control on entropy.
Top-k	Keep the k highest-probability tokens; a fixed cut independent of shape.
Total variation distance	Half the L1 distance between two distributions, in [0, 1].

27.39 Further Reading

Primary sources, verified on 1 August 2026 at `platform.claude.com/docs`:

- `build-with-claude/working-with-messages` — the basic request, the sampling-parameter restriction verbatim, mid-conversation system messages, prefill and vision.
- `about-claude/models/migration-guide` — *"the SDK request types still define these fields… but the API rejects the request server-side"*, and `max_tokens` as a hard limit on thinking plus text.
- `build-with-claude/effort` and `build-with-claude/task-budgets` — the five levels, the default, the per-model support and the advisory contract.
- `build-with-claude/structured-outputs` — the format control that replaced low-temperature format tricks.
- `build-with-claude/fast-mode` and `api/service-tiers` — `speed` and `service_tier`.
- `api/errors` — the 400 shapes this chapter's preflight reproduces.

Where to go next in this book. Chapter 28 takes the response apart — every `stop_reason`, including the `max_tokens` case Panel D generates so many of. **Chapter 29** turns Chapter 26's retry arithmetic and this chapter's 400s into an error-handling discipline. **Chapter 30** covers Batch, prompt caching and cost controls, where `max_tokens` and `effort` become budget instruments rather than shape controls. Further back, **Chapter 3** is the softmax this chapter made quantitative and **Chapter 13** is the test-time-compute argument that `effort` operationalises; further forward, **Chapter 54** treats `context_management` in full.